

bashqueues

Job Queue Management System

User Manual

Version 0.17.90 · May 2026

System users · administrators · security professionals

Contents

1 Overview and Concepts

- 1.1 What bashqueues does
- 1.2 Key terms
- 1.3 Job lifecycle

2 System User Guide

- 2.1 Submitting jobs
- 2.2 Scheduling and dependencies
- 2.3 Monitoring jobs
- 2.4 Controlling jobs
- 2.5 Draft jobs
- 2.6 Cron bridge
- 2.7 Command reference — user

3 Administrator Guide

- 3.1 Installation and setup
- 3.2 Job classes
- 3.3 Asset plugins
- 3.4 Execution caps
- 3.5 Workers and runners
- 3.6 Execution environments (env profiles)
- 3.7 Health, backup, and diagnostics
- 3.8 Multi-user and system daemon
- 3.9 Reporting and FinOps
- 3.10 Command reference — admin

4 Security Professional Guide

- 4.1 Security architecture
- 4.2 Sandbox levels
- 4.3 Seccomp profiles
- 4.4 Runtime caps
- 4.5 Class-statement policies
- 4.6 Authorisation codes and code signing
- 4.7 Exception overlays
- 4.8 Static security audit (secaudit)
- 4.9 Clearance audit trail
- 4.10 Interrogation profiles
- 4.11 Governance: jurisdiction and classification
- 4.12 Command reference — security

PART 1

Overview and Concepts

1.1 What bashqueues does

bashqueues is a shell-native job queue. It accepts work items called *jobs*, stores them on the local filesystem, and runs them sequentially or in parallel according to operator-defined rules. A worker process polls the queue and launches the next eligible job. The system requires no network service, no daemon started at boot, and no database — the queue is a directory.

Jobs carry resource limits, sandboxing constraints, retry policies, scheduling windows, dependency chains, security authorisations, and behavioural profiles. All are recorded in the job file and verified before execution. A sentinel process handles cheap control-plane work independently of workers.

1.2 Key terms

- **Job** — a named work item with a command, priority, and metadata. Stored as a plain text .job file.
- **QID** — the queue ID. A unique, time-based identifier (e.g. 20260525_103044_000123_4521).
- **Class** — a configuration template applied to jobs sharing the same processing rules.
- **Asset** — a named runtime condition checked before a job starts. Assets block dispatch if unsatisfied.
- **Cap** — an execution cap plugin enforcing limits on a running job's behaviour.
- **Worker** — a process that polls the queue and launches eligible pending jobs.
- **Sentinel** — a lightweight control-plane process that handles policy gating, stale-job cleanup, and deadline escalation without launching payloads.
- **Clearance** — the recorded moment a job passed all policy and preflight checks and was approved for execution.
- **Interrogation profile** — a captured behavioural baseline (syscalls, network, file access) used to gate future runs.
- **Policy** — a site-wide or class-level rule governing sandbox levels, security exceptions, and authorisation requirements.

1.3 Job lifecycle

State	Meaning
pending	Waiting to be picked up by a worker.
running	Currently executing.
done	Completed successfully (exit code 0).
failed	Completed with a non-zero exit code, or retries exhausted.
paused	Manually held; not dispatched until unpaused.
pol_block	Blocked by policy at submission or dispatch. Requires authorisation or policy change.
interrupted	Worker detected stale. Resubmit if the command is safe to retry.
cancelled	Operator-terminated. ON_FAILURE hooks are not fired.
deleted	Soft-deleted; recoverable with queue undelete.

PART 2

System User Guide

For operators and developers who submit and monitor jobs.

2.1 Submitting jobs

```
# Basic submission
queue submit my-job -- /opt/scripts/process.sh

# With priority, retries, resource limits
queue submit my-job --priority 5 --retries 3 --backoff 60 \
  --cpu 50 --mem 512M -- /opt/scripts/process.sh

# Assign to a class
queue submit my-job --class OVERNIGHT_WINDOW -- /opt/scripts/etl.sh

# With sandbox and seccomp
queue submit my-job --sandbox network-none --seccomp docker-default \
  -- /opt/scripts/safe.sh

# Preview without submitting
queue submit --dryrun my-job -- /opt/scripts/process.sh

# JSON output for programmatic use
queue submit my-job --json -- /opt/scripts/process.sh
```

--priority N / -p N	Integer priority. Lower = higher. Default 10.
--retries N	Maximum automatic retries on failure.
--backoff SEC	Seconds between retry attempts.
--cpu PCT	CPU quota percentage. Enforced by systemd.
--mem SIZE	Memory limit (e.g. 256M, 1G). Enforced by systemd.
--class NAME	Job class. Inherits class defaults.
--runner auto systemd direct	Execution backend.
--sandbox LEVEL	Sandbox level: off, network-none, restrict-egress, strict.
--seccomp PROFILE	Seccomp profile: off, docker-default, strict.
--after-success QID	Dependency: do not start until named job succeeds.
--on-success CMD...	Run when this job completes successfully.
--on-failure CMD...	Run when this job fails permanently.
--dryrun / -n	Show what would happen without writing anything.
--json / -j	Emit JSON output for programmatic use.

2.2 Scheduling and dependencies

```
# Submit with a delay
queue submit-in 2h my-job -- /opt/scripts/cleanup.sh
queue submit-in 30m my-job -- /opt/scripts/poll.sh

# Submit at a wall-clock time
queue submit-at 23:30 my-job -- /opt/scripts/nightly.sh
queue submit-at '2026-06-01 08:00' my-job -- /opt/scripts/monthly.sh

# Dependency chain
queue submit export-data -- /opt/scripts/export.sh
queue submit archive-data --after-success export-data \
  -- /opt/scripts/archive.sh

# Show scheduled and waiting jobs
queue scheduled
queue waiting
```

2.3 Monitoring jobs

```
# List jobs (all states or filtered)
queue list
queue list --state pending
queue list --state failed --json

# Full detail on a specific job
queue explain <QID>
queue explain <QID> --json

# Show job and log tail
queue show <QID>
queue show <QID> --full

# Follow live log
queue tail <QID>
queue tail <QID> --no-follow

# Statistics
queue stats
queue stats --today

# Process IDs for running job
queue pids <QID>

# Resource metrics (systemd runner)
queue metrics <QID>

# Recent audit events
queue events
queue events --tail 50

# Clearance audit (jobs that passed dispatch checks)
queue cleared
queue cleared --json --state done,failed --since '2026-05-01'
queue audit cleared --limit 100
```

2.4 Controlling jobs

queue cancel <QID>	# Cancel pending or running job
queue pause <QID>	# Hold a pending job
queue unpause <QID>	# Release a paused job
queue resubmit <QID>	# Re-queue failed or interrupted job
queue priority <QID> 3	# Change priority
queue delete <QID>	# Soft-delete

```
queue undelete <QID>           # Recover soft-deleted job
queue on-success <QID> -- /opt/scripts/notify.sh
queue on-failure <QID> -- /opt/scripts/alert.sh
```

TIP

Cancel does not fire ON_FAILURE hooks. Use cancel when you are certain the command should not be retried. For jobs that should be retried later, use pause instead.

2.5 Draft jobs

```
queue draft create my-draft --class OVERNIGHT_WINDOW \
  -- /opt/scripts/etl.sh --full
queue draft list
queue draft show my-draft
queue draft ready my-draft      # Mark as ready for review
queue draft submit my-draft     # Submit into the queue
queue draft create-from-job <QID>
```

2.6 Cron bridge

The cron bridge reads crontab-format files and submits due entries as queue jobs. Each unique command gets a generated class with CLASS_MAX_CONCURRENT=1, preventing overlapping runs — the fundamental problem with traditional cron. The class selector plugin automatically assigns an appropriate class when none is declared.

```
queue cron list                # List registered crontabs
queue cron explain [user]      # Explain entries for a user
queue cron tick --dryrun       # Preview what would fire now
queue cron preview --now 23:45 # Preview at a specific time
queue cron edit                # Edit personal crontab
queue cron show                # Show personal crontab
```

```
# Crontab format
# minute hour day month weekday command
*/15 * * * * /opt/scripts/poll.sh
0 6 * * 1-5 /opt/scripts/morning-report.sh
```

```
# Force a specific class
# BASHQUEUES_CLASS=BATCH_PROCESSING
0 22 * * * /opt/scripts/nightly-etl.sh
```

NOTE

@daily, @weekly, @hourly and similar shorthand macros are supported. @reboot is not. When no class is declared, the selector plugin analyses the command and assigns the most appropriate bundled class automatically.

2.7 Command reference — user

<code>queue submit NAME [opts] -- CMD</code>	Submit a job.
<code>queue submit-in DELAY NAME -- CMD</code>	Submit with a delay (30s, 10m, 2h, 1d).
<code>queue submit-at TIME NAME -- CMD</code>	Submit at a wall-clock time.
<code>queue list [--state S] [--name N] [--json]</code>	List jobs.
<code>queue show QID [--full]</code>	Show job record and log tail.
<code>queue explain QID [--json]</code>	Full job detail with policy and exceptions.
<code>queue tail QID [--no-follow]</code>	Follow live job log.
<code>queue pids QID</code>	Show process IDs for a running job.
<code>queue metrics QID</code>	Show systemd resource metrics.
<code>queue stats [--today]</code>	Count jobs in each state.
<code>queue events [--tail N]</code>	Show recent audit events.
<code>queue cleared [--json] [--since DATE]</code>	Show jobs that passed dispatch clearance.
<code>queue cancel QID</code>	Cancel a pending or running job.
<code>queue pause / unpause QID</code>	Hold or release a pending job.
<code>queue resubmit QID</code>	Re-queue a failed or interrupted job.
<code>queue delete / undelete QID</code>	Soft-delete or recover a job.
<code>queue priority QID N</code>	Change a job's priority.
<code>queue on-success QID -- CMD</code>	Set success hook.
<code>queue on-failure QID -- CMD</code>	Set failure hook.
<code>queue history [NAME]</code>	Execution history for a job name.
<code>queue deps QID</code>	Show dependency status.
<code>queue waiting / scheduled</code>	List jobs waiting on deps or schedule.
<code>queue draft list / submit NAME</code>	Manage draft jobs.
<code>queue cron list / explain / tick</code>	Cron bridge commands.
<code>queue version</code>	Print installed version.

PART 3

Administrator Guide

For queue owners and system administrators.

3.1 Installation and setup

```
# Add to ~/.bashrc
source /opt/bashqueues/queuebash.sh

# Override queue root (default ~/.queuebash)
export QUEUEBASH_ROOT=/var/lib/myapp/queue

# Start workers
queue run                # Foreground, single worker
queue start --workers 4  # 4 detached background workers
queue daemon             # Sentinel + 1 worker (shorthand)

# Sync bundled classes, assets, caps, policies from source
queue classes refresh /opt/bashqueues/classes
queue assets refresh /opt/bashqueues/assets.d
queue caps refresh /opt/bashqueues/caps.d
queue policies refresh /opt/bashqueues/policies.d
```

3.2 Job classes

```
# Minimal class
CLASS_ALLOW_PARALLEL=1
CLASS_MAX_CONCURRENT=2
CLASS_DEFAULT_RUNNER=auto
CLASS_DEFAULT_SANDBOX_LEVEL=network-none

# With assets and scheduling window
queue_class_shared_asset path freespace /var/lib/output min_mb=500
queue_class_shared_asset service active postgresql.service
queue_class_shared_asset time window overnight-window \
  weekdays=mon-fri weekday_windows=18:00-05:00 \
  weekends=sat-sun weekend_windows=always
```

queue classes list	List all installed classes.
queue class show NAME	Print the class file.
queue class init NAME	Create a minimal class skeleton.
queue class edit NAME	Open the class in \$EDITOR.
queue class validate [NAME]	Check class syntax.
queue class explain NAME	Show class with resolved defaults.
queue class replace NAME FILE	Hot-swap a class file.
queue class rollback NAME	Restore the previous class version.
queue classes refresh DIR	Sync all classes from a directory.

NOTE

Class defaults are snapshotted into each job file at submission time. Changing a class does not affect in-flight or already-submitted jobs.

3.3 Asset plugins

Family	Facilities	Purpose
db	postgres_connect, mysql_connect, sqlite_accessible, redis_connect	Database connectivity.
deadline	monitor, panic	Deadline escalation and panic fallback. Evaluated by sentinel.
disk	smart_ok, md_healthy, md_degraded, mount_writable, io_error_count	Drive health and RAID state.
docker	running, not_running, healthy, image_exists, image_age, no_privileged	Container state. Supports Docker and Podman.
finops	spot_price_below, budget_remaining, anomaly_free	Cost control gates from local pricing/budget caches.
format	json, xml, yaml, csv, archive	File format validation.
git	repo_exists, clean_tree, branch	Git repository state.
k8s	reachable, deployment_ready, pod_running, no_crashlooping, resource_quota_ok	Kubernetes cluster and workload state.
net	http_status, tcp_endpoint, interface_state, allowance	Network reachability and usage.
path	exists, mount, freespace	Filesystem path and space.
proc	running, not_running, pid_file, max_instances, cpu_user, mem_user	Process presence and resource usage.
queue	no_running, pending_count, last_success_age, queue_depth	Queue self-referential checks.
runnable	script, interpreter, path, env_var, resource	Execution environment checks.
secaudit	script_safe, no_destructive, no_network_c2, no_privesc, no_obfuscation	Static security analysis.
service	active, healthy, failed, property	systemd/SysV service state.
snmp	truth_ok	SNMP NMS maintenance window check.
sys	memory_available, cpu_load, cpu_cores, iowait	System resource levels.
time	window	Time-of-day and weekday scheduling windows.
vm	running, not_running, snapshot_exists, pool_active	libvirt/KVM virtual machine state.

3.4 Execution caps

Family	Caps	Purpose
billing	cpu_seconds, net_usage_mb	Accounting caps.
net_usage	egress_mb, ingress_mb	Network traffic limits.
runtime	no_spawn_shell, no_network_tools, no_network_sockets, only_local_sockets, only_port	Runtime security caps via /proc and lsof.

NOTE

Runtime caps sample the job process tree at `RUNTIME_CAP_INTERVAL` seconds (default 1). This is a security-relevant parameter: classes with security caps should keep it at 1. lsof must be installed for socket-based caps.

3.5 Workers and runners

Runner	Behaviour	Notes
auto	Systemd when available, direct otherwise.	Recommended for most installations.
systemd	Transient systemd user unit. Enables resource enforcement and sandboxing.	Required for CPU/memory limits.
direct	Direct shell execution. No resource enforcement.	Environments without systemd.

```

queue run                # Foreground worker
queue start --workers 2  # Detached workers
queue daemon             # Sentinel + 1 worker
queue sentinel --interval 30 --detach # Sentinel only
queue workers            # List active workers
queue limits             # Check resource limit support
queue limits --probe     # Test with a real systemd probe

```

3.6 Execution environments (env profiles)

Execution environment profiles declare chroot boundaries, secret scopes, endpoint scopes, and data classification for jobs. A class assigns a default environment via `CLASS_EXEC_ENV=NAME`. Jobs can request a specific environment at submission time.

```

queue env list           # List installed env profiles
queue env show NAME     # Show profile fields
queue env validate NAME  # Validate profile syntax
queue env path NAME     # Show profile file path

```

```

# Example profile fields (envs.d/staging.env)
EXEC_ENV_NAME=staging
EXEC_ENV_LABEL=Staging environment
EXEC_ENV_CHROOT_MODE=off
EXEC_ENV_SECRET_SCOPE=staging
EXEC_ENV_ENDPOINT_SCOPE=staging
EXEC_ENV_DATA_SCOPE=test
EXEC_ENV_CONFIRMATION_REQUIRED=0

```

NOTE

Profiles are sourced by the worker before launching the payload. EXEC_ENV_CHROOT_ROOT with systemd creates a filesystem namespace boundary. EXEC_ENV_CONFIRMATION_REQUIRED=1 halts dispatch until an operator confirms.

3.7 Health, backup, and diagnostics

```
queue health                # Full health report
queue health --fix          # Auto-fix stale running jobs
queue stats                 # Job counts by state
queue dispatch-trace        # Recent dispatch decisions

# Backup and restore queue state
queue backup create          # Snapshot queue root to archive
queue backup restore         # Restore from archive

# Log management
queue compress-logs         # Bulk-compress completed logs
queue clean-logs --older-than 30d

# Plugin verification
queue plugins verify         # Verify loaded plugin signatures
queue plugins audit         # Inventory installed plugins
queue code verify           # Verify queuebash.sh signature
queue code audit            # Show code signing policy
```

3.8 Multi-user and system daemon

```
# Operate on another user's queue (root only)
queue user alice list
queue user alice explain <QID>
queue user alice exception list <QID>
queue queue-users           # List users with queues

# System-wide daemon for all users
queue system-daemon         # Foreground
queue system-daemon --detach # Background

# Sentinel – cheap control-plane, no payload execution
queue sentinel --interval 30 --detach
queue scheduler --interval 30 # Alias
queue supervise --interval 30 # Alias
```

IMPORTANT

queue user selects a different queue root. Job scripts evaluate with the issuing user's privileges, not the queue owner's. This is enforced by the foreign-user guard built into the command dispatcher.

3.9 Reporting and FinOps

Reporters are plugin scripts in reporters.d/ that emit structured summaries from job history. The FinOps asset family gates dispatch on locally-cached pricing and budget data, preventing cost-sensitive workloads from running during anomalous conditions.

```
# Reporting
queue reporting list        # List installed reporter plugins
queue reporting run NAME    # Run a reporter
```

```

queue reporting run NAME --json # Machine-readable output

# FinOps asset usage in a class
queue_class_shared_asset finops anomaly_free _ block_on=error missing=ok
queue_class_shared_asset finops spot_price_below "0.08" \
  instance_type=m5.large region=us-east-1
queue_class_shared_asset finops budget_remaining "monthly-ml" \
  min_remaining=20

# FinOps health stream (fed by external tooling)
# $QUEUEBASH_ROOT/streams/finops.health

```

3.10 Command reference — administrator

<code>queue start --workers N</code>	Start N background workers.
<code>queue daemon</code>	Sentinel + 1 worker.
<code>queue sentinel --interval N</code>	Start control-plane sentinel.
<code>queue workers</code>	List active workers.
<code>queue limits [--probe]</code>	Check resource limit support.
<code>queue health [--fix]</code>	Health report with optional repair.
<code>queue backup create / restore</code>	Snapshot or restore queue state.
<code>queue classes refresh DIR</code>	Sync classes from a directory.
<code>queue assets refresh DIR</code>	Sync asset plugins.
<code>queue caps refresh DIR</code>	Sync cap plugins.
<code>queue policies list</code>	List installed policies.
<code>queue policies show KIND NAME</code>	Show a policy file.
<code>queue policies edit KIND NAME</code>	Edit a policy file.
<code>queue env list / show / validate</code>	Manage execution environment profiles.
<code>queue global claims</code>	Show cross-user resource claims.
<code>queue modules list / enable</code>	Manage installed modules.
<code>queue user NAME CMD</code>	Run a command against another user's queue.
<code>queue queue-users</code>	List users with queues.
<code>queue reporting list / run</code>	Reporter plugins.
<code>queue dispatch-trace</code>	Recent dispatch decisions.
<code>queue plugins verify / audit</code>	Verify plugin signatures.
<code>queue code verify / audit</code>	Verify source code signatures.
<code>queue compress-logs</code>	Bulk-compress job logs.
<code>queue mgr</code>	Open interactive queue manager panel.

PART 4

Security Professional Guide

Sandboxing, policies, authorisation, profiling, and audit.

4.1 Security architecture

bashqueues implements layered, defence-in-depth job containment. Each layer is independent: defeating one does not disable the others.

Layer	Timing	Mechanism
1 — Static audit	Submission	secaudit asset checks for dangerous patterns before the job file is written.
2 — Class policy	Submission	Class-statement policy gates block or divert jobs based on user, command hash, sandbox level, and authorisation.
3 — Asset preflight	Pre-dispatch	Asset checks gate execution on runtime conditions. Job stays pending until all pass.
4 — Sandbox	Execution	systemd unit properties (PrivateNetwork, ProtectHome, etc.) constrain what the job process can access.
5 — Seccomp filter	Execution	Kernel syscall filters derived from the declared seccomp profile.
6 — Runtime caps	During execution	Worker samples /proc and lsof. Violations terminate the job.
7 — Interrogation profile	Dispatch gate	Signed behavioural baseline. Unsigned or mismatched profiles block dispatch when enforced.
8 — Exception overlays	Operator grant	Per-job overrides requiring a reason or authorisation code. Every application is audited.

4.2 Sandbox levels

Level	Properties	Use case
off	No systemd sandbox properties.	Trusted internal scripts.
network-none	PrivateNetwork=yes.	Most batch processing.
restrict-egress	Network restricted to declared outbound.	Jobs needing local IPC.
strict	PrivateNetwork, ProtectHome=read-only, PrivateTmp, ProtectSystem=strict.	Maximum isolation.

4.3 Seccomp profiles

Profile	Filter	Use case
docker-default	Denies @clock, @debug, @module, @mount, @privileged, @raw-io, and other sensitive groups.	Most service and processing workloads.
strict	Allows only @system-service. Denies everything else.	Maximum restriction.

```
# Apply at submission
queue submit my-job --seccomp docker-default -- CMD

# Declare in class
CLASS_DEFAULT_SECCOMP_PROFILE=docker-default

# Allow an additional syscall group (exception, requires reason)
queue submit my-job --seccomp strict --seccomp-allow @debug \
  --reason 'strace debugging run' -- CMD
```

4.4 Runtime caps

no-spawn-shell	Terminate if a child shell (sh, bash, zsh, ksh) is spawned.
no-network-tools	Terminate if curl, wget, nc, socat, or similar is executed.
no-network-sockets	Terminate if any INET socket is opened.
only-local-sockets	Terminate if any socket is bound to a non-localhost address.
only-port	Terminate if a TCP/UDP port outside the allow-list is opened.

```
# Enable in class
CLASS_DEFAULT_RUNTIME_CAPS=no-spawn-shell,only-local-sockets
CLASS_DEFAULT_RUNTIME_CAP_INTERVAL=1
CLASS_DEFAULT_RUNTIME_CAP_PORTS=5432,6379,8000-8099
```

4.5 Class-statement policies

```
# policies.d/class-statement/default.env (key variables)
CLASS_POLICY_USER_SANDBOX_POLICIES="off network-none restrict-egress strict queue-default"
CLASS_POLICY_USER_SECCOMP_POLICIES="off docker-default strict queue-default"
CLASS_POLICY_EXCEPTION_FLAGS_REQUIRE=reason-or-authorisation
CLASS_POLICY_AUTHORISATION_SIGNATURE_REQUIRED=if-trusted-key

# Per-user grants
CLASS_POLICY_USER_ALICE_SANDBOX_GRANT=strict

# List / edit / create policies
queue policies list
queue policies show class-statement default
queue policies edit class-statement default
queue policies create class-statement site-custom
```

4.6 Authorisation codes and code signing

```
# Generate a signing key pair (Ed25519)
queue keygen authorisation OPS_KEY

# Generate an authorisation code for a specific command
queue authorisation generate \
  --admin ops-team --user alice --expires 2026-06-30 \
  --reason 'emergency recovery run' \
```

```
-- /opt/scripts/recovery.sh --full

# Submit using the code
queue submit recovery \
  --authorisation AUTH-XXXX-YYYY-ZZZZ \
  -- /opt/scripts/recovery.sh --full

# List and inspect codes
queue authorisation list
queue authorisation show AUTH-XXXX-YYYY-ZZZZ
queue authorisation policy

# Code signing for queuebash.sh and plugins
queue code sign --file queuebash.sh
queue code verify
queue code audit
queue plugins verify
queue keys list
```

NOTE

Authorisation codes bind to the command by SHA256 hash. Changing a single argument byte invalidates the code. This prevents an approved code from being reused for a different invocation.

4.7 Exception overlays

```
# Add an exception: bypass the time window check for this job
queue exception add <QID> time:window \
  --reason 'emergency daytime run approved by ops lead'

# Target: family, facility, or full family:facility:target
queue exception add <QID> time                # whole family
queue exception add <QID> time:window          # facility
queue exception add <QID> time:window:overnight-window # specific target

queue exception list <QID>
queue exception list-all
queue exception clear <QID> time:window
```

IMPORTANT

Exceptions for sandbox, seccomp, runtime caps, and port restrictions are the highest-privilege overrides in the system. Every application is written to events.jsonl with the operator identity and timestamp.

4.8 Static security audit (secaudit)

Facility	Params	Detects
secaudit:no_destructive	allow_rm=0	rm -rf wipes, mkfs, dd disk overwrites, fork bombs, critical device overwrites.
secaudit:no_network_c2		/dev/tcp reverse shells, nc listener/exec patterns, bash -i >&, curl-pipe-shell.
secaudit:no_privesc	allow_sudo=0	sudo, su -, chmod 777/SUID, chown root.
secaudit:no_obfuscation		eval \$VAR, base64-decode-pipe-to-shell.
secaudit:script_safe	strict=0	All four checks on a script file. strict=1 also runs shellcheck.
secaudit:string_safe	strict=0	All four checks on a literal command string.

4.9 Clearance audit trail

Every job that passes all policy and preflight checks is stamped with a *clearance record* — the moment, worker identity, active policy snapshot, sandbox level, seccomp profile, runtime caps, and any security exemption details. Clearance records are queryable independently of job state and survive queue clear via the archive.

```
# Query clearance records
queue cleared
queue cleared --json
queue cleared --state done,failed --since 2026-05-01 --limit 200
queue audit cleared --json

# Clearance fields (JSON)
# id, name, class, policy_statement, sandbox, seccomp, runtime_caps,
# exemption_type, exemption_action, exemption_code,
# jurisdiction, classification, cleared_at, worker, stage

# events.jsonl records complementary job events:
queue events --tail 100
```

4.10 Interrogation profiles

Interrogation profiles capture the actual behaviour of a command — which syscalls it makes, which network hosts it contacts, which files it reads and deletes — under strace and lsof observation. The compiled profile is then reviewed, approved, and signed. Subsequent submissions can be gated on a valid signed profile.

The signing model is currently tamper-evident: a SHA256 hash is computed over the approved profile content and stored with the operator identity and timestamp. Integration with the existing Ed25519 key infrastructure (used by authorisation codes) is the next planned step.

```
# 1. Profile a command
queue profile interrogate run my-script -- /opt/scripts/process.sh
queue profile interrogate run my-script --pre-arm-delay 0.75 -- CMD

# 2. Campaign mode (multiple runs for delayed or stateful behaviour)
queue profile interrogate repeat my-script --count 10 -- CMD
queue profile interrogate diff-runs ~/.queuebash/profiles/.../<campaign-id>
queue profile interrogate merge ~/.queuebash/profiles/.../<campaign-id> \
  --name my_script

# 3. Compile from a single run
queue profile interrogate compile ~/.queuebash/profiles/.../<run-id> \
  --name my_script

# 4. Review
queue profile interrogate explain my_script

# 5. Approve and sign
queue profile interrogate approve my_script
queue profile interrogate approve my_script --kind seccomp \
  --signing-key ops-key --accept-warnings

# 6. Show approved profiles
queue profile interrogate show my_script

# 7. Compare a new run against the approved baseline
queue profile interrogate diff my_script ~/.queuebash/profiles/.../<new-run-id>
```

IMPORTANT

Profile compilation is PID-tree scoped. Machine-wide network context (from ss/lsof system-wide samples) is recorded as context fields but is intentionally excluded from generated allow-lists to prevent unrelated background traffic from polluting the policy. Only lsof -p against the profiled process tree is used for network policy candidates.

The explain output highlights risk blockers and warnings that require operator acknowledgement:

Blocker/Warning	Trigger	Operator action needed
destructive_syscalls_require_review	unlink, unlinkat, rmdir, or similar syscalls observed.	Operator must confirm why destructive operations are expected.
network_egress_requires_review	Outbound network connections observed in the PID tree.	Operator must confirm the remote host and purpose.
opened_listener	Process opened a TCP/UDP listener.	Requires explicit justification; unusual for batch scripts.
global_network_context_ignored	Machine-wide network activity was recorded but excluded from policy.	Informational only; not a blocker.
privileged_context	Profile was captured as root or in an elevated context.	Profile may not match non-privileged production execution.

4.11 Governance: jurisdiction and classification

Jobs can carry jurisdiction and data classification metadata. These fields are recorded in the clearance record and the job file, and are available to reporter plugins and the JSON API.

```
# Submit with governance metadata
queue submit my-job \
  -- env QUEUEBASH_JURISDICTION=EU QUEUEBASH_CLASSIFICATION=restricted \
    /opt/scripts/gdpr-export.sh

# Governance fields appear in queue cleared --json output:
# "governance": {
#   "jurisdiction": "EU",
#   "classification": "restricted"
# }

# Gate dispatch on classification with the secaudit family
queue_class_shared_asset secaudit no_network_c2 \
  "${QUEUEBASH_EXPORT_SCRIPT}"
queue_class_shared_asset secaudit no_privesc \
  "${QUEUEBASH_EXPORT_SCRIPT}"
```

4.12 Command reference — security

<code>queue submit --sandbox LEVEL</code>	Submit with sandbox level.
<code>queue submit --seccomp PROFILE</code>	Submit with seccomp profile.
<code>queue submit --sandbox-override LEVEL</code>	Override class sandbox (exception).
<code>queue submit --seccomp-allow GROUP</code>	Allow extra syscall group (exception).
<code>queue submit --drop-cap CAP</code>	Remove a runtime cap (exception).
<code>queue submit --add-port PORT</code>	Add an allowed port (exception).
<code>queue submit --reason TEXT</code>	Provide reason for security exception.
<code>queue submit --authorisation CODE</code>	Attach a signed authorisation code.
<code>queue keygen authorisation NAME</code>	Generate Ed25519 signing key pair.
<code>queue keys list</code>	List installed public keys.
<code>queue authorisation generate ...</code>	Generate an authorisation code.
<code>queue authorisation list</code>	List active codes.
<code>queue authorisation show CODE</code>	Inspect an authorisation record.
<code>queue authorisation policy</code>	Show active authorisation policy.
<code>queue exception add QID ASSET</code>	Add an asset exception overlay.
<code>queue exception list QID</code>	List exceptions for a job.
<code>queue exception list-all</code>	All jobs with exceptions.
<code>queue exception clear QID ASSET</code>	Remove a specific exception.
<code>queue policies list / show / edit</code>	Manage security policies.
<code>queue profile interrogate run NAME</code>	Profile a command once.
<code>queue profile interrogate repeat NAME</code>	Campaign profiling (N runs).
<code>queue profile interrogate compile DIR</code>	Compile traces into candidates.
<code>queue profile interrogate explain NAME</code>	Review risk blockers and warnings.
<code>queue profile interrogate approve NAME</code>	Approve and sign a profile.
<code>queue profile interrogate diff NAME DIR</code>	Compare run against approved baseline.
<code>queue code sign / verify / audit</code>	Code signing for source files.
<code>queue plugins verify / audit</code>	Plugin signature verification.
<code>queue cleared [--json]</code>	Clearance audit query.
<code>queue events [--tail N]</code>	Audit event log.